IV FCS HSE Scientific Conference (2026)

A sound pure type system with ranks as sorts and Type-in-Type

Pavel Sokolov[1] Alexander Kuklev[2]

[1]HSE University, Moscow

[2]Radboud University, Nijmegen

Historical Background

Pure type systems (PTS) are a family of minimalistic typed λ-calculi introduced independently by Berardi and Terlouw. Their grammar is given in Fig. 1.

Variable	x	$\in$	$\mathcal{X}$	
Sort	s	$\in$	$\mathcal{S}$	
Term	T, U, t, u	$::=$	x	variable
		$\mid$	s	sort as type
		$\mid$	$\Pi(x\colon T).U$	type of dependent functions
		$\mid$	$\lambda(x\colon T).u$	anonymous function
		$\mid$	$u\,t$	function application

Figure 1: Grammar of pure type systems

Typing rules of a single PTS (given in Fig. 2) are determined by two relations:

- *axiom* relation $\mathsf{Ax} \subseteq \mathcal{S} \times \mathcal{S}$ determines "to which sort(s) a given sort belongs as an element".
- *rule* relation $\mathsf{Rule} \subseteq \mathcal{S}^2 \times \mathcal{S}$ places types of functions $\Pi(x : T).U$ into sorts accounting for the sorts where T and U are situated.

$$\textsc{Ax}\ \frac{}{\Gamma \vdash s : s'}(s, s') \in \mathsf{Ax} \qquad \textsc{Rule}\ \frac{\Gamma \vdash T : s_T \qquad \Gamma, x : T \vdash U : s_U}{\Gamma \vdash \Pi(x : T).U : s}(s_T, s_U, s) \in \mathsf{Rule}$$

$$\textsc{Var}\ \frac{}{\Gamma \vdash x : T}(x : T) \in \Gamma \qquad \textsc{Abs}\ \frac{\Gamma \vdash T : s_T \qquad \Gamma, x : T \vdash u : U}{\Gamma \vdash \lambda(x : T).u : \Pi(x : T).U}$$

$$\textsc{App}\ \frac{\Gamma \vdash u : \Pi(x : T).U \qquad \Gamma \vdash t : T}{\Gamma \vdash u\,t : U[t/x]} \qquad \textsc{Conv}\ \frac{\Gamma \vdash t : T \qquad \Gamma \vdash U : s \qquad T =_\beta U}{\Gamma \vdash t : U}$$

Figure 2: Typing rules of pure type systems

The main result in the theory of PTS is that some of them are actually non-normalizing.

Theorem (Girard'72 [1])

The following PTS called System U is not normalizing:

$$\mathcal{S} = \{*, \square, \triangle\}, \quad \mathsf{Ax} = \{(*, \square), (\square, \triangle)\},$$
$$\mathsf{Rule} = \{(*, *, *), (\square, *, *), (\triangle, *, *), (\square, \square, \square), (\triangle, \square, \square)\}$$

Furthermore, the offending term has type $\Pi(P : *).P$.

This result is dubbed "Girard's paradox" since it renders System U inconsistent with respect to its intended logical semantics: inhabitedness of $\Pi(P : *).P$ would mean that every proposition P is provable. Thus, any system where System U embeds is inconsistent, too; for example, the following PTS known as "Type-in-Type" is inconsistent:

$$\mathcal{S} = \{*\}, \quad \mathsf{Ax} = \{* : *\}, \quad \mathsf{Rule} = \{(*, *, *)\}.$$

Long story short, it motivates an introduction of a *universe hierarchy* $\mathcal{S} = \{\mathcal{U}_k\}_{k\in\mathbb{N}}$ with axioms $\mathcal{U}_k : \mathcal{U}_{k+1}$ and rules $(\mathcal{U}_k, \mathcal{U}_l, \mathcal{U}_{k\vee l})$ for each k, l; when interpreted in sets, $[\![\mathcal{U}_k]\!]$ is taken to be the k-th inaccessible cardinal [3].

Our system

We present the following PTS which we dub PTS-rk:

$$\mathcal{S} = \{\mathsf{rk}_r\}_{r\in\mathbb{N}} \quad \mathsf{Ax} = \{(\mathsf{rk}_r, \mathsf{rk}_0)\}_{r\in\mathbb{N}} \quad \mathsf{Rule} = \{(\mathsf{rk}_r, \mathsf{rk}_s, \mathsf{rk}_{(r+1)\vee s})\}_{r,s\in\mathbb{N}}$$

(Naming "rk" comes from "rank", since rules compute precisely the rank of functions, which is the maximal nesting of Π in the domain type.)

Main results

- PTS-rk has a self-containing sort $\mathsf{rk}_0 : \mathsf{rk}_0$ but Girard's paradox doesn't apply.
- PTS-rk is strongly normalizing; we proved it in axiom-free Rocq.
- Normalization of rank-bounded terms is $\mathrm{ELEMENTARY}$.
- Typechecking is TOWER-complete.
- All elementary recursive functions are computable in PTS-rk.
- PTS-rk has a model in ZF+GCH.
- PTS-rk fully interprets Tarski–Mostowski–Robinson arithmetic.

Proof Ideas

Note that greedy reduction of lambda terms increases term exponentially and decreases maximal rank of redex arguments. This explains normalization and typechecking behaviour; upper bound on computable functions follows from bound on the size of normal form. Representability of elementary recursive functions follows from representability of basis $(+, \times, \uparrow, \mathrm{mod})$ — a corresponding substitution-basis theorem can be found in [2]. Set-theoretic model uses $\aleph$-s to encode dependent function types; GCH is needed to validate rules of PTS-rk. Interpretation of arithmetic uses double negation to validate classical principles and domain predicates are used to run induction when necessary.

Examples

Numeral-preserving lowering, exponentiation and increment modulo 5:

$$\delta_j \triangleq \lambda\, n\, X\, f\, x.\, n\, (X \to X)\, (\lambda\, h\, y.f\, (h\, y))\, (\lambda\, y.y)\, x : \mathcal{N}_{j+1} \to \mathcal{N}_j$$
$$\mathsf{Exp}_j \triangleq \lambda\, m\, n\, (X : \mathsf{rk}_j)\, f\, x.\, n\, (X \to X)\, (m\, X)\, f\, x : \mathcal{N}_j \to \mathcal{N}_{j+1} \to \mathcal{N}_j$$
$$c_5 \triangleq \lambda\, s\, X\, x_1\, x_2\, x_3\, x_4\, x_5.\, s\, X\, x_2\, x_3\, x_4\, x_5\, x_1 : V_5 \to V_5$$

Some logical connectives and propositional equality on numerals:

$$\bot \triangleq \Pi(Z : \mathsf{rk}_0).Z \quad : \quad \mathsf{rk}_1$$
$$\wedge \triangleq \lambda\, P\, Q.\, (P \to Q \to \bot) \to \bot \quad : \quad \mathsf{rk}_j \to \mathsf{rk}_j \to \mathsf{rk}_{j+2}$$
$$\equiv \triangleq \lambda\, m\, n.\, \Pi(P : \mathcal{N}_1 \to \mathsf{rk}_1).\ \neg(P\, m) \leftrightarrow \neg(P\, n) \quad : \quad \mathcal{N}_1 \to \mathcal{N}_1 \to \mathsf{rk}_5$$

*

References

Antonius J. C. Hurkens. "A Simplification of Girard's Paradox". In: *Typed Lambda Calculi and Applications (TLCA 1995)*. Vol. 902. Lecture Notes in Computer Science. Springer, 1995, pp. 266–278. DOI: 10.1007/BFb0014058.

Mihai Prunescu, Lorenzo Sauras-Altuzarra, and Joseph M. Shunia. *A Minimal Substitution Basis for the Kalmár Elementary Functions*. 2026. arXiv: 2505.23787 [math.LO]. URL: https://arxiv.org/abs/2505.23787.

Benjamin Werner. "Sets in Types, Types in Sets". In: *Theoretical Aspects of Computer Software (TACS 1997)*. Vol. 1281. Lecture Notes in Computer Science. Springer, 1997, pp. 530–546. DOI: 10.1007/BFb0014566.

arxiv.org/abs/2610.25085

github.com/TurtlePU/pts-rk